

Docker Bind Mount — Complete Summary

1. Named Volume

A named volume is managed by Docker. Docker chooses where the data is physically stored on the host.

```
docker volume create todo-db
docker volume ls
docker volume inspect todo-db

docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started
```

Key idea: todo-db is the volume name and /etc/todos is the container mount point.

2. Bind Mount

A bind mount connects a specific folder on the host machine to a location inside the container. You control the host path.

```
-v HOST_PATH:CONTAINER_PATH

Example:
-v "$(pwd):/app"
```

Named volume: Docker chooses the host location.

Bind mount: You choose the host location.

3. Why Bind Mounts?

Bind mounts are very useful during development. Your source code can stay on your Windows machine while the container uses that same code. Tools such as nodemon can detect changes and restart the application.

```
Windows source code
↓
Bind mount
↓
Container /app
↓
nodemon detects changes
↓
Application restarts
```

4. PowerShell Development Command

```
docker run -dp 3000:3000 `
  -w /app `
  -v "$(pwd):/app" `
  node:18-alpine `
  sh -c "yarn install && yarn run dev"
```

-w /app sets the container's working directory. **-v "\$(pwd):/app"** bind-mounts the current host folder to /app.

5. Test a Bind Mount

```
docker exec <container-id> sh -c "echo HelloFromContainer > /app/test.txt"
docker exec <container-id> cat /app/test.txt

docker stop <container-id>
docker rm <container-id>
```

Because /app is bind-mounted, test.txt appears in your Windows project folder and remains there after the container is deleted.

6. Persist the Todo Database with a Bind Mount

The Todo application's SQLite database is stored at `/etc/todos/todo.db`. Mounting only `/app` does not persist this database. Therefore, use a second bind mount for `/etc/todos`.

```
docker run -dp 3000:3000 `
  -w /app `
  -v "$(pwd):/app" `
  -v "$(pwd)/todo-data:/etc/todos" `
  node:18-alpine `
  sh -c "yarn install && yarn run dev"
```

The first mount is for source code. The second mount is for database data.

```
Windows app folder      → /app
Windows app/todo-data   → /etc/todos
                        ■■■ todo.db
```

7. Test Todo Persistence

1. Start the container with the two bind mounts.
2. Open `http://localhost:3000`.
3. Add tasks such as `t1`, `t2`, `t3`.
4. Stop and remove the container.
5. Recreate it using the same two bind mounts.
6. The tasks should still be present because `todo.db` is stored in the host's `todo-data` folder.

```
docker ps
docker logs -f <container-id>

docker stop <container-id>
docker rm <container-id>

docker run -dp 3000:3000 `
  -w /app `
  -v "$(pwd):/app" `
  -v "$(pwd)/todo-data:/etc/todos" `
  node:18-alpine `
  sh -c "yarn install && yarn run dev"
```

8. Useful Docker Commands

Purpose	Command
List running containers	<code>docker ps</code>
List all containers	<code>docker ps -a</code>
Stop container	<code>docker stop <container-id></code>
Remove container	<code>docker rm <container-id></code>
View logs	<code>docker logs -f <container-id></code>
Run command inside container	<code>docker exec <container-id> <command></code>
Create named volume	<code>docker volume create todo-db</code>
List volumes	<code>docker volume ls</code>
Inspect volume	<code>docker volume inspect todo-db</code>

9. Final Concept to Remember

```
Named Volume
-v todo-db:/etc/todos
  ↑           ↑
volume name   container path
Docker manages host location

Bind Mount
```

```
-v "${pwd}/todo-data:/etc/todos"
  ↑           ↑
host folder/path container path
YOU control host location
```

One-line explanation: A named volume is managed by Docker, while a bind mount connects a specific host folder to a location inside the container. Bind mounts are especially useful for development because host source-code changes are immediately available inside the container.